

# An Analysis of Gradient descent

Gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

## Gradient descent -- Part 1

reformulated as a quadratic minimization problem. If the system matrix  $\mathbf{A}$  is real symmetric and positive-definite, an objective function is defined as the quadratic function, with minimization of

## Gradient descent -- Part 2

Because each step is taken in the steepest direction, steepest-descent steps alternate between directions aligned with the extreme axes of the elongated level sets. When  $\kappa(\mathbf{A})$  is large, this produces a characteristic zig-zag path. The poor conditioning of  $\mathbf{A}$  is the primary cause of the slow convergence, and orthogonality of successive residuals reinforces this alternation. As shown in the image on the right, steepest descent converges slowly due to the high condition number of  $\mathbf{A}$ , and the orthogonality of residuals forces each new direction to undo the overshoot from the previous step. The result is a path that zigzags toward the solution. This inefficiency is one reason conjugate gradient or preconditioning methods are preferred.

## Gradient descent -- Part 3

as in the Barzilai-Borwein method, or a sequence  $\eta_n$  satisfying the Wolfe conditions (which can be found by using line search). When the function  $f$  is convex, all local minima are also global minima, so in this case gradient descent can converge to the global solution. This process is illustrated in the adjacent picture. Here,  $f$  is assumed to be defined on the plane, and that its graph has a bowl shape. The blue curves are the contour lines, that is, the regions on which the value of  $f$  is constant. A red arrow originating at a point shows the direction of the negative gradient at that point. Note that the (negative) gradient at a point is orthogonal to the contour line going through that point. We see that gradient descent leads us to the bottom of the bowl, that is, to the point where the value of the function  $f$  is minimal.

## Gradient descent -- Part 4

$\mathbf{r} := \mathbf{b} - \mathbf{A} \mathbf{x}$  repeat in the loop:  $\eta := \mathbf{r}^T \mathbf{r} / \mathbf{r}^T \mathbf{A} \mathbf{r}$   $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$  if  $\|\mathbf{r}\|$  is sufficiently small, then exit loop  $\mathbf{r} := \mathbf{r} - \eta \mathbf{A} \mathbf{r}$  end repeat

```
loop return x as the result
\begin{aligned} \mathbf{r} &:= \mathbf{b} - \mathbf{A} \mathbf{x} \\ \&\&\{\text{repeat in the loop:}\} \&\& \eta := \mathbf{r}^T \mathbf{r} / \mathbf{r}^T \mathbf{A} \mathbf{r} \\ \&\& \&\& \mathbf{x} &:= \mathbf{x} + \eta \mathbf{r} \\ \&\& \&\& \mathbf{r} &:= \mathbf{r} - \eta \mathbf{A} \mathbf{r} \\ \&\& \&\& \&\& \{\text{if } \|\mathbf{r}\| \text{ is sufficiently small, then exit loop}\} \\ \&\& \&\& \&\& \mathbf{r} &:= \mathbf{r} - \eta \mathbf{A} \mathbf{r} \\ \&\& \&\& \&\& \&\& \{\text{return } \mathbf{x} \text{ as the result}\} \end{aligned}
```

## Gradient descent -- Part 5

$\mathbf{J}_G(\mathbf{0}) = \begin{bmatrix} 3 & 0 & 0 & 0 & 2 & 0 & 0 & 2 \end{bmatrix}$ ,  $\mathbf{G}(\mathbf{0}) = \begin{bmatrix} -2.5 & -1 & 10.472 \end{bmatrix}$ .  
 $\mathbf{J}_G(\mathbf{0}) = \begin{bmatrix} 3 & 0 & 0 & 0 & 2 & 0 & 0 & 2 \end{bmatrix}$ ,  $\mathbf{G}(\mathbf{0}) = \begin{bmatrix} -2.5 & -1 & 10.472 \end{bmatrix}$ .

## Gradient descent -- Part 6

The method is rarely used for solving linear equations, with the conjugate gradient method being one of the most popular alternatives. The number of gradient descent iterations is commonly proportional to the spectral condition number  $\kappa(\mathbf{A})$  of the system matrix  $\mathbf{A}$  (the ratio of the maximum to minimum eigenvalues of  $\mathbf{A}$ ), while the convergence of conjugate gradient method is typically determined by a square root of the condition number, i.e., is much faster. Both methods can benefit from preconditioning, where gradient descent may require less assumptions on the preconditioner.

## Gradient descent -- Part 7

$f(\mathbf{x}(1)) \leq f(\mathbf{x}(0)) = f(\mathbf{0})$ .  
 $f(\mathbf{x}(1)) \leq f(\mathbf{x}(0)) = f(\mathbf{0})$ .

## Gradient descent -- Part 8

$\nabla f(\mathbf{x}) = 2 \mathbf{A} (\mathbf{A} \mathbf{x} - \mathbf{b})$ .  
 $\nabla f(\mathbf{x}) = 2 \mathbf{A} (\mathbf{A} \mathbf{x} - \mathbf{b})$ .

## Gradient descent -- Part 9

Using gradient descent in C++, Boost, Ublas for linear regression  
Series of Khan Academy videos discusses gradient ascent  
Online book teaching gradient descent in deep neural network context  
Archived at Ghostarchive and the Wayback Machine: "Gradient Descent, How Neural Networks Learn". 3Blue1Brown. October

# An Analysis of Gradient descent

Gradient descent

16, 2017 – via YouTube. Garrigos, Guillaume; Gower, Robert M. (2023). "Handbook of Convergence Theorems for (Stochastic) Gradient Methods". arXiv:2301.11235 [math.OC].

## Gradient descent -- Part 10

Forgo the benefits of a clever descent direction by setting  $\mathbf{p}_n = \nabla f(\mathbf{a}_n)$ , and use line search to find a suitable step-size  $\gamma_n$ , such as one that satisfies the Wolfe conditions. A more economic way of choosing learning rates is backtracking line search, a method that has both good theoretical guarantees and experimental results. Note that one does not need to choose  $\mathbf{p}_n$  to be the gradient; any direction that has positive inner product with the gradient will result in a reduction of the function value (for a sufficiently small value of  $\eta_n$ ). Assuming that  $f$  is twice-differentiable, use its Hessian  $\nabla^2 f$  to estimate  $\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n) \approx -t\eta_n \nabla^2 f(\mathbf{a}_n) \mathbf{p}_n$ . Then choose  $\mathbf{p}_n$  and  $\eta_n$  by optimising inequality (1). Assuming that  $\nabla f$  is Lipschitz, use its Lipschitz constant  $L$  to bound  $\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n) \approx -t\eta_n \nabla^2 f(\mathbf{a}_n) \mathbf{p}_n$ . Then choose  $\mathbf{p}_n$  and  $\eta_n$  by optimising inequality (1). Build a custom model of  $\max_{t \in [0, 1]} \nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n) \approx -t\eta_n \nabla^2 f(\mathbf{a}_n) \mathbf{p}_n$  for  $f$ . Then choose  $\mathbf{p}_n$  and  $\eta_n$  by optimising inequality (1). Under stronger assumptions on the function  $f$  such as convexity, more advanced techniques may be possible. Usually by following one of the recipes above, convergence to a local minimum can be guaranteed. When the function  $f$  is convex, all local minima are also global minima, so in this case gradient descent can converge to the global solution.